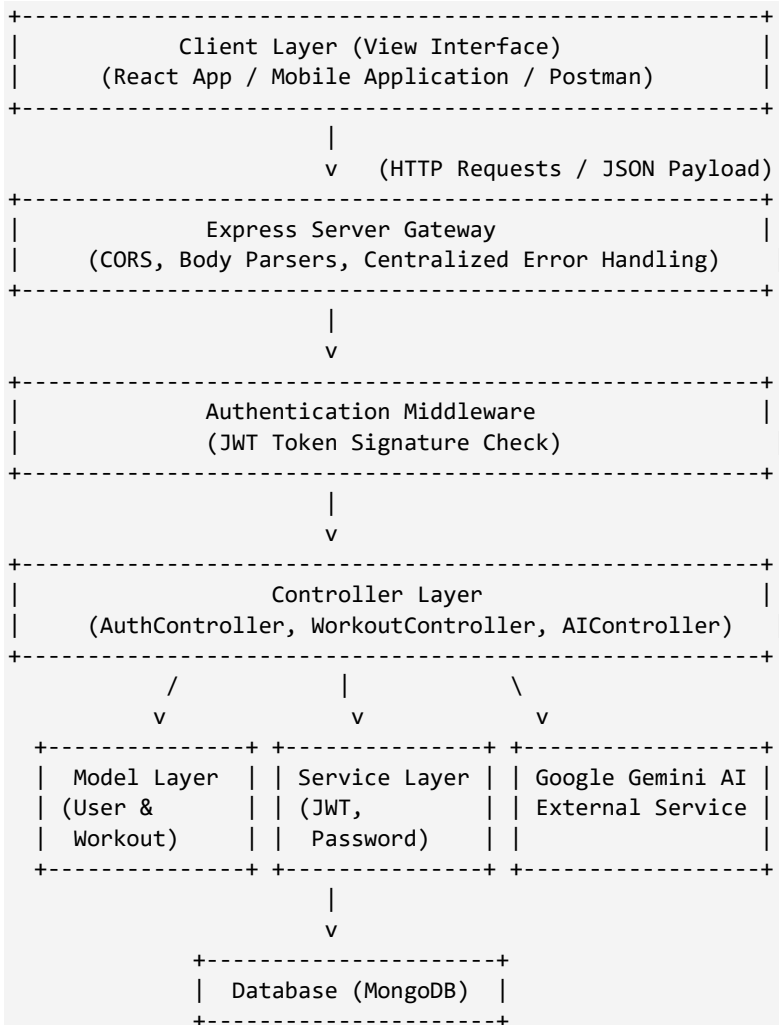


PROJECT DESIGN PHASE

1. Technical Architecture

The FitTrack AI backend follows the Model-View-Controller (MVC) architectural design pattern, augmented with a dedicated Service Layer and Middleware Layer to promote modularity, maintainability, and loose coupling between components.



Layer Breakdown & Responsibilities

- Model Layer (Data Layer)**: Defines the data structure, constraints, and validation rules using Mongoose Schemas; communicates directly with MongoDB to perform persistent database operations; contains the User Schema (credentials and profile data) and the Workout Schema (activity tracking).
- View Layer (API Interface)**: As a RESTful backend API, the application does not serve traditional server-side HTML UI templates. The View Layer is represented by API consumers such as React frontend web apps, mobile applications, or testing utilities like Postman / Thunder Client. Receives input via HTTP request payloads and consumes standard JSON responses.
- Controller Layer (Business Logic)**: Acts as the intermediary between routes, services, and models. Validates input data, executes business rules, calls database methods or external services, and formats standard JSON responses. Includes authController, workoutController, and aiController.
- Service Layer (Reusable Modules)**: Houses isolated, highly reusable business logic used across multiple controllers. Gemini AI Service prepares prompts and sends demographic or activity stats to Google Gemini API, returning structured workout plans and fitness insights. JWT Service handles signing and verifying JSON Web Tokens. Password Hashing Service manages bcrypt.js salt generation, hashing, and comparison.

2. ER-Diagram

The application backend manages data persistence through two primary collections connected via a One-to-Many relationship.

A. User Entity (users collection)

Stores registered user credentials and profile metadata.

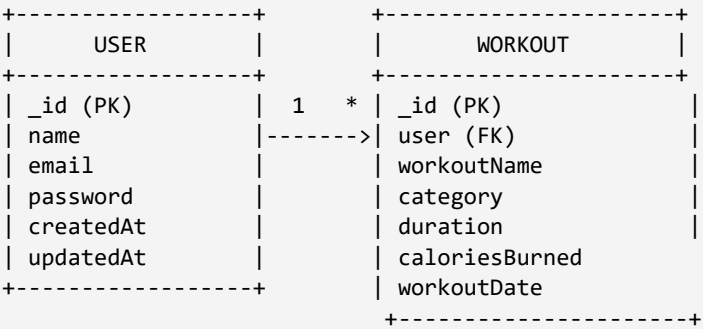
Field	Type	Constraint	Description
_id	ObjectId	Primary Key, Auto-generated	Unique document identifier
name	String	Required	Full name of the user
email	String	Required, Unique, Lowercase	User email address for authentication
password	String	Required, Encrypted (bcrypt.js)	Hashed user password
createdAt	Date	Timestamp (Auto)	Record creation timestamp
updatedAt	Date	Timestamp (Auto)	Record last update timestamp

B. Workout Entity (workouts collection)

Stores individual workout session logs logged by authenticated users.

Field	Type	Constraint	Description
_id	ObjectId	Primary Key, Auto-generated	Unique document identifier
user	ObjectId	Foreign Key (ref: 'User'), Required	References the parent User document
workoutName	String	Required	Name of exercise (e.g., "Morning Run")
category	String	Required	Category (e.g., Cardio, Strength)
duration	Number	Required, min: > 0	Exercise duration in minutes
caloriesBurned	Number	Required, min: >= 0	Total estimated calories burned
workoutDate	Date	Required	Date when the workout occurred
createdAt	Date	Timestamp (Auto)	Record creation timestamp
updatedAt	Date	Timestamp (Auto)	Record last update timestamp

C. Entity Relationship: One-to-Many (User → Workouts)



- Relationship Explanation: One User can log multiple Workout records over time. However, each Workout document belongs strictly to one parent user, enforced by referencing the user’s ObjectId in the user field.

3. Features

The key capabilities delivered by the FitTrack AI backend:

- Secure Authentication — Registration and login protected by bcrypt.js password hashing and JWT-based session tokens.
- Workout Management — Full CRUD (Create, Read, Update, Delete) for personal workout logs, scoped to the authenticated user.
- Workout Search — Dynamic filtering of workout history by keyword, category, or date.
- Google Gemini Integration — A dedicated service layer wraps the @google/genai SDK for all AI calls.
- AI Workout Recommendations — Personalized weekly routines generated from age, fitness goal, and experience level.
- AI Fitness Insights — Progress evaluation and motivational feedback generated from aggregated workout statistics.
- Centralized Error Handling — A single Express error-handling middleware standardizes every failure response.
- RESTful, Modular Architecture — Routes, controllers, services, and models are cleanly separated for maintainability and parallel development.

4. Roles and Responsibilities

The application currently defines a single application-level role — the Authenticated User — enforced by the JWT authentication middleware on every protected route.

Authenticated User

Responsibilities:

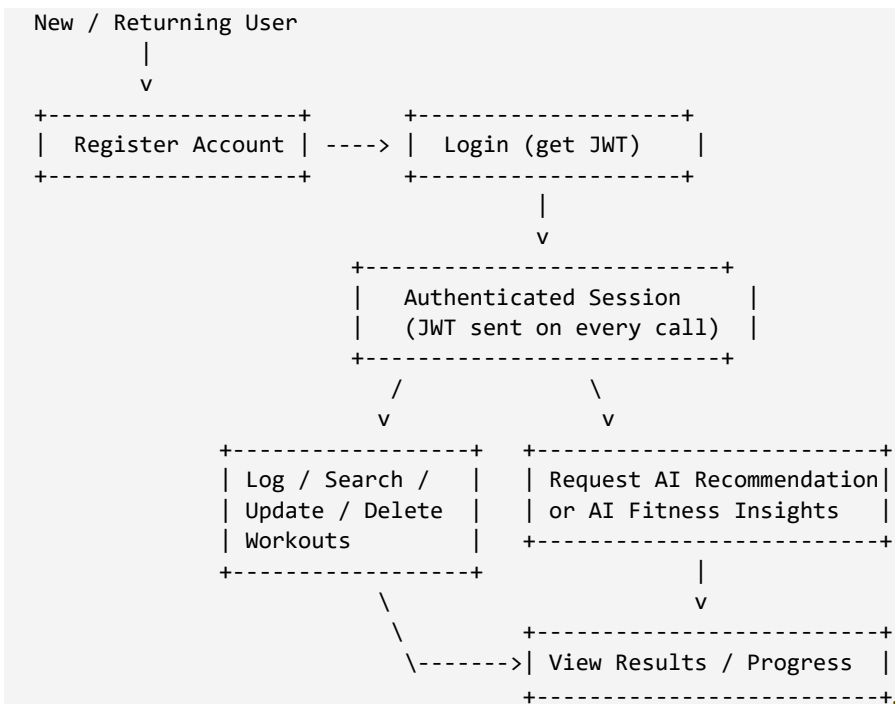
- Register an account and keep login credentials secure.
- Log workouts accurately (name, category, duration, calories, date).
- Review AI-generated recommendations and insights and apply them responsibly.

Permissions:

- Full CRUD access to their own workout records only — every query is scoped by req.user.id, so one user can never read or modify another user's data.
- Read access to their own profile via GET /api/auth/profile.
- Access to both AI endpoints (recommendation and insights) using their own workout statistics.

5. User Flow

End-to-end path a user takes through the system, from first visit to receiving an AI insight:



6. MVC Pattern

FitTrack AI applies the MVC pattern as follows, expanding on the layer breakdown above:

Model Layer (Data Layer)

- Mongoose schemas (User, Workout) define shape, types, and validation rules for every document stored in MongoDB.
- Models are the only layer that talks directly to the database.

View Layer (API Interface)

- There is no server-rendered HTML — the "view" is the JSON contract returned to API consumers (a future React client, a mobile app, or Postman during testing).
- Every response follows the standardized { success, data } / { success, error } shape defined in NFR3.

Controller Layer (Business Logic)

- authController, workoutController, and aiController each own one functional domain.
- Controllers validate input, call the Service or Model layer, and translate the result into an HTTP response — they never talk to MongoDB directly.

7. API Route Mapping

The API routes are categorized by functional domain, mapped to specific HTTP methods, and secured using the JWT authentication middleware where applicable.

A. Auth Routes (/api/auth)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/auth/register	Public	registerUser	Creates a new user account with hashed password.
POST	/api/auth/login	Public	loginUser	Verifies password & returns JWT authorization token.

Method	Route Endpoint	Access	Controller Method	Description
GET	/api/auth/profile	Protected (JWT)	getUserProfile	Retrieves profile info of the logged-in user.

B. Workout Routes (/api/workouts)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/workouts/	Protected (JWT)	addWorkout	Logs a new workout session linked to the user.
GET	/api/workouts/	Protected (JWT)	getWorkouts	Fetches all workouts belonging to the user.
GET	/api/workouts/search	Protected (JWT)	searchWorkouts	Filters workouts by keyword, category, or date.
GET	/api/workouts/:id	Protected (JWT)	getWorkoutById	Retrieves a specific workout document by ID.
PUT	/api/workouts/:id	Protected (JWT)	updateWorkout	Updates an existing workout record.
DELETE	/api/workouts/:id	Protected (JWT)	deleteWorkout	Removes a workout record permanently.

C. AI Integration Routes (/api/ai)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/ai/recommendation	Protected (JWT)	getAIRecommendation	Generates weekly exercise plan using Gemini AI.
POST	/api/ai/insights	Protected (JWT)	getFitnessInsights	Generates performance analysis from workout stats.